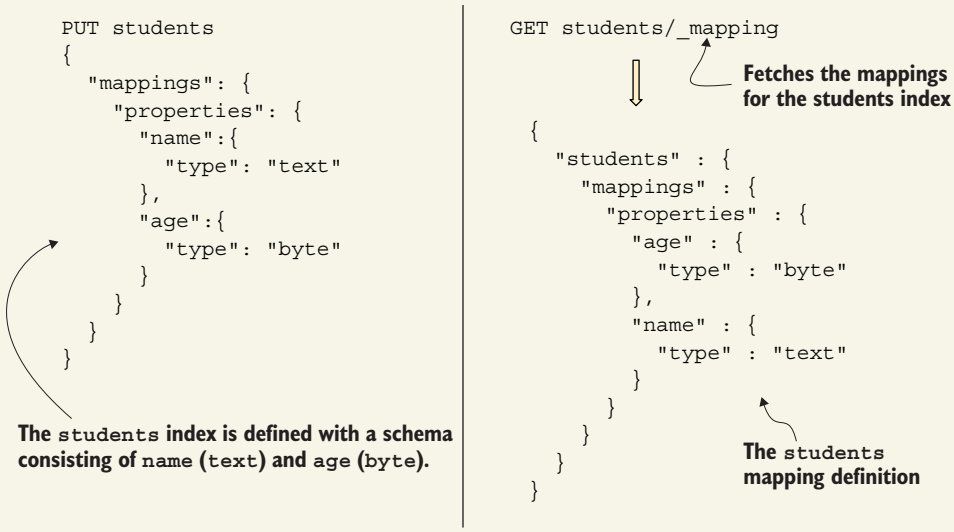
In the next few sections, we review these data types, their characteristics, and their usage. But before we work with data types in detail, we need to understand how to create mapping definitions. Section 4.3 digs deeper into working with mapping features, but for now we briefly look at a mechanism to create a mapping definition as it is a prerequisite for the next sections.

Developing mapping schemas

When we indexed a movie document (listing 4.1), Elasticsearch created a schema dynamically (figure 4.2), deriving types by analyzing field values. We are now at a point where we would like to create our schema definitions. Elasticsearch uses an indexing API to create these definitions, so let's check it out with an example.

Suppose we wish to create a student with `name` and `age` fields that have the data type `text` and `byte`, respectively (the next section introduces these data types). We can create the `students` index with these mapping definitions up front, thus taking control of creating the schema. The figure demonstrates this succinctly, with the mapping definition execution on the left-hand side and fetching the definition on the right-hand side.



Mapping definition (left) and retrieving the schema (right)

We create the index with a request containing a set of field properties wrapped in a `mappings` object. We invoke the `_mapping` endpoint on the index to check the definition of the schema (shown on the right-hand side).

Now that we know how to create schemas at a high level, we are ready to examine the core data types.